Lavendra Boutique — SPA Mini Project Report

CS3404: GUI Programming — Vue 3 / TypeScript / Tailwind CSS SPA (DummyJSON API)

Author: Uvin Nanayakkara | University of Ruhuna, Faculty of Engineering | EG/2022-2023

1. Project Overview

Lavendra Boutique is a data-driven e-commerce single page application built with Vue 3 (Composition API), TypeScript and Tailwind CSS. It consumes the public DummyJSON products API and reproduces a bespoke Figma design: a navy/lavender boutique storefront with a hero banner, curated collection filters, a product grid, an exclusive deals rail and a newsletter footer.

2. Features Implemented

- Product catalogue fetched live from GET /products (DummyJSON), typed end-to-end with TypeScript interfaces — no `any` types.
- Collection filter bar ("All Collections", "Tech Essentials", "Modern Apparel", "Home Office", "Limited Drop", "Architectural Prints") mapping the boutique's own naming onto real DummyJSON category slugs.
- Live text search across product title, category and brand.
- Product detail view reachable via a dynamic route (/product/:id) — Vue Router bonus feature.
- Persistent shopping cart built on a Pinia store that survives page reloads via localStorage — Cart/Bookmarks bonus feature.
- Wishlist (heart) toggle per product, also persisted to localStorage.
- Dark mode toggle using Tailwind's `dark:` class strategy, remembering the user's preference — Dark Mode bonus feature.
- Exclusive Deals section and a dedicated Deals page, sorted by steepest discount from the live API data.
- Fully responsive layout (mobile / tablet / desktop) using Tailwind Grid and Flex utilities.
- Loading skeletons and empty-state messaging for network/filter states.

3. Component Architecture

The application is deliberately split into small, single-purpose components rather than one monolithic App.vue. State is centralised in three Pinia stores (products, cart, wishlist, theme) so components stay presentational and reusable.

```
App.vue
■■■ NavBar.vue                (routes, theme toggle, cart badge)
■■■ RouterView
■   ■■■ HomeView.vue
■   ■   ■■■ Hero.vue
■   ■   ■■■ CollectionFilter.vue → SearchBar.vue
■   ■   ■■■ ProductGrid.vue → ProductCard.vue → StarRating.vue
```

```
■   ■   ■■■ DealsSection.vue → DealCard.vue
■   ■   ■■■ Newsletter.vue
■   ■■■ CategoriesView.vue    (CollectionFilter.vue + ProductGrid.vue)
■   ■■■ DealsView.vue         (DealCard.vue list)
■   ■■■ ProductDetailView.vue (StarRating.vue, quantity + add-to-cart)
■   ■■■ CartView.vue          (cart line items, subtotal, checkout stub)
■■■ AppFooter.vue

stores/  products.ts · cart.ts · wishlist.ts · theme.ts
api/     products.ts   (typed fetch wrappers around DummyJSON)
types/   product.ts    (Product, ProductListResponse, Collection, ...)
```

4. Data & State Flow

On mount, HomeView/CategoriesView/DealsView call the shared **products** Pinia store, which fetches from DummyJSON once and caches the result. Collection pills and the search box mutate reactive filter state on that same store; a getter (*filteredProducts*) recomputes the visible list. Adding an item to the cart or wishlist dispatches an action on the respective store, which updates reactive state and mirrors it to localStorage so the cart survives a refresh.

5. Tech Stack

Layer	Choice
Framework	Vue 3 (Composition API, <script setup>)
Language	TypeScript (strict)
Build tool	Vite
Styling	Tailwind CSS 3 (class-based dark mode)
State management	Pinia
Routing	Vue Router 4 (dynamic /product/:id route)
Data source	DummyJSON REST API (fetch)

See prompts.txt for the AI-usage log and README.md for setup/run instructions.